

COSC 3P32 – Introduction to Database Systems

Winter 2026

Group Project, Part 2

Due Date: 30th March, 2026, 1:59pm

No late assignments are accepted.

This work accounts for 12% of your final grade and is worth a total of 120 marks.

Please pay special attention to the due date (and time) of this project. This coincides with the start of the **first lab of the week** (i.e. for most people, this will be before your own lab day/time).

The project report and all work must be completed by the due date.

Demonstrations must take place during the lab in which you are registered.

In this part of your project, you will build the database that you designed in Part 1, and demonstrate different ways of interacting with the database. You must use PostgreSQL in conjunction with Microsoft Access. All work should be completed using your group account.

A. Implementation: 100 marks

Create tables and insert data

Use the DB schema obtained from Part 1 of the project to create the tables required for your application. If you received a poor mark for your design from Part 1, or are unhappy with your design, then you can discuss this with the instructor or TAs and you can choose to use as a starting point the ER diagram and list of relations provided by the instructor.

Input relevant data into your tables. You should input enough data that it is possible to verify all of your queries, along with all of the constraints that must hold.

Notes:

You should carefully review the `TIMESTAMP` type to understand how it works. You are allowed to use an alternate format (e.g. break down timestamp into day, month, year, hour, minute, and second, with each represented as an integer). You do not need to worry about split-seconds. Also, you should investigate the use of `VARCHAR` vs. `CHAR`.

As with any implementation of the SQL standard, some features will not be supported or will have some slight variation. For the list of unsupported features in PostgreSQL, see:

<https://www.postgresql.org/docs/12/unsupported-features-sql-standard.html>

Enforce constraints

You must ensure that all constraints specified in Part 1 are enforced, as well as those already specified in Assignment 1. Some of these constraints can only be enforced via check constraints and/or triggers – note that assertions are not available.

Total participation constraints are very difficult to enforce unless they are also combined with a 1-many relationship (i.e. with the corresponding tables being combined). As a result, they appear in the “difficult requirements” section below.

Implement functionality

The basic requirements for this project are listed below and **must** be implemented but you are not limited to this functionality. You may add your own functionality (e.g. additional forms) as you think of things that may provide a more user-friendly environment. You may create views to simplify some of the queries if you wish.

Create a project that includes **forms** to support all of the following:

- Add/delete/modify information about an account and the viewers for that account.
- Add/delete/modify information about a video (movie, show, or other) and its director.
- Add/delete/modify information about an actor, including information on the videos in which they have acted.

The project must also support all of the following **queries**:

- Given the title and year of a movie, output the following information on the actors in that movie: stage name, pay, and role for the movie.
- Given the title and year of a show, output all information on all episodes of that show.
- Given an account ID, output the viewer name for all viewers belonging to that account, together with their watch history for a given calendar month: for each watch interval, the start and end of the interval and the title and year of the video watched during that interval. For the purposes of testing, ensure that you at least have enough data (and variety of data) to demonstrate correctness of this query for each of January, February, and March 2026.
- Given the stage name of an actor, find the impact of that actor on different genres of movies, defined as follows: for each genre of movie in which they have acted, the average rating of all of those movies.
- Find the top 5 videos for a given calendar month, defined as follows: the title and year of the video, together with the total number of different viewers who have watched that video in that month. Note that some viewers may have watched the video multiple times, or during multiple intervals; in such cases, only one such interval is to be counted regardless of its length. For the purposes of testing, ensure that you at least have enough data to demonstrate correctness of this query for each of January, February, and March 2026.

Important note concerning queries: obviously the ideal solution is to provide attractive reports for each of the above queries, however if necessary you can use a workaround as follows:

- Create a view that implements the query for all possible data, then in PostgreSQL write a simple query on the view for the given input.
- For example, for the first query listed above the view would be defined for all possible movies, and the simple query to be written in PostgreSQL queries that view for a given movie.

How to get a good mark – more difficult requirements: a “bare bones” project that fulfills all of the above requirements will receive at most 80 marks out of 100. To receive the additional 20 marks for the implementation, you must implement some or all of the following. Be warned that some may be very difficult, so you should concentrate on the rest of your project first.

- Nice layouts for your forms
- Creation of attractive reports instead of the workaround with views/queries
- Implementation of a start-up menu for your application, from which the user may select all of the options (functionality defined above) that they wish to use.
- Enforcement of a total participation constraint (enforcing just *one* will suffice). You may find enforcing such a constraint within Access is easier than enforcing it for any kind of insertion, deletion, or update. But this means that a person inserting/deleting/updating by other means

(e.g. using PostgreSQL directly) could be able to get around the constraint. Any attempt at enforcing such a constraint could warrant some bonus marks, but be warned that it is difficult.

- Anything that provides a more user-friendly environment (e.g. functionality and appearance).

B. Documentation: 10 marks

By the due date, you must submit on Brightspace a report including all of the following:

- A full listing (saved to PDF) of your entire group project directory. This must list *all* database files used in this project, including the date/time on which they were saved. This must be produced immediately prior to submitting your project.
- The database schema.
- A list of all data for each of your tables.
- Report on division of labour within your group. Reminder: unless there are exceptional circumstances, all members of the same group will receive the same mark. If the instructor has been informed that there are indeed exceptional circumstances, then this report may be used to adjust relative marks within a group.

Be sure to spend adequate time preparing the report as this is one of the main things the TA will look at to mark your project.

Important note about your group account:

After the demonstration, your group account password will be reset so that the TA can access it for marking purposes. Also, no modifications are allowed to your project after submission, unless requested by the TA for demonstration purposes. Therefore you should make sure that everything required to run your project is *in your group account prior to submission*.

C. Demonstration: 10 marks

Your project must be demonstrated to the TA in the lab in which you are registered during the week of 30th March.

All group members are expected to be present at the demonstration. The group members must be able to walk through the functionality of the application.

At the demonstration, the TA will participate in the walkthrough of your application by viewing the data, providing appropriate input and asking questions. The TA may write additional queries, etc. at this time to test your application. Due to time constraints it will not be possible to fully test the entire application during the demonstration, but the TA will look for specific functionality. The entire application will be marked after the demonstration, by access to the group account as specified above.

Submission Requirements:

1. You must submit a report on Brightspace as specified above. As noted, pay special attention to the date and time at which it is due, which in most cases is on an earlier date than that of your lab/demonstration. Only one submission is needed per group.
2. No changes to any of the files linked to this project are allowed after the time at which the project is submitted (except as requested by the TA for demonstration purposes). Any such changes will result in an automatic grade of ZERO for this project.